

文献地图（按主题分类）

执行反馈与可验证奖励 (Execution Feedback & Verifiable Reward)

- **CodeRL: Mastering Code Generation through Pretrained Models and Deep RL (NeurIPS 2022)** – Hung Le et al (Salesforce) ¹ 提出利用**预训练模型+深度RL**进行代码合成。它将代码生成模型视为actor，引入一个critic预测功能正确性，从**单元测试反馈**提供密集奖励 ²。训练用CodeT5改进架构并在APPS数据上取得SOTA性能 ³。要点：利用测试信号训练actor-critic，**解决了监督训练忽略测试反馈的问题**。
- **Execution-based Code Generation using Deep RL (PPOCoder, arXiv 2023)** – Parshin Shojaei et al. 利用**PPO算法**优化代码生成，以执行结果为奖励 ⁴。该方法将传统PPO应用于代码合成任务，用单元测试通过情况作为奖励信号，**奠定了代码生成RL训练的基础** ⁴。
- **RLTF: Reinforcement Learning from Unit Test Feedback (TMLR 2023)** – Jiatae Liu et al (Tencent) ⁵ 提出**在线RL+细粒度测试反馈**框架。训练过程中实时生成数据，并利用**多粒度单元测试信号**（包括错误定位等）指导模型生成更高质量代码 ⁵。在APPS、MBPP上达到SOTA性能。要点：**利用详细的执行反馈（如出行行号）而非仅通过/未通过二元信号，提高了训练效率**。
- **RLEF: Grounding Code LLMs in Execution Feedback with RL (arXiv 2024)** – Jonas Gehring et al (Meta) ⁶ 提出**RLEF方法**，将RL用于代码的自我纠错迭代。模型在**多轮推理**中根据执行反馈进行调整 ⁷。结果表明，引入执行反馈的RL训练使LLM能**解决更复杂、长程的编码任务** ⁷。
- **VerPO: Verifiable Dense Reward Policy Optimization for Code Generation (arXiv 2026)** – Longwen Wang et al (TeleAI) ⁸ ⁹ 专注于**奖励设计**。指出传统**全或无（二元）奖励信号**稀疏，导致梯度消失 ¹⁰；而学到的奖励模型易出现偏差和开销大 ¹¹。VerPO提出通过**部分单测通过率赋予加权稠密奖励**，动态估计每个测试难度权重并累加作为奖励，同时结合最终全通过的全局奖励以保持一致性 ¹² ¹³。在HumanEval、BigCodeBench等多基准上，VerPO比仅二元奖励或学得奖励模型的基线**提升pass@1最多8.8%** ⁹。要点：**利用可验证的执行反馈构造连续奖励，缓解稀疏和误导问题**。
- **CodePRM: Execution Feedback-Enhanced Process Reward Model (ACL 2025)** – Qingyao Li et al. ¹⁴ 提出**训练过程级奖励模型（PRM）**，利用执行轨迹对代码推理过程打分，提供比最终结果更细粒度的监督。在无高质量测试时，PRM可看作一种**自动评价器**，用于指导RL训练。然而作者也指出，纯靠学得奖励可能存在**对齐偏差**，需配合可验证信号使用 ¹⁵。

多轮代码生成与自我修复 (Multi-turn Code Generation & Self-Repair)

- **μCode: Multi-Turn Code Generation Through Single-Step Rewards (ICML 2025)** – Arnav Jain et al (Cornell & AWS) ¹⁶ 提出**μCode框架**解决多轮代码生成的信用分配难题。关键洞见是：代码生成是“一步可恢复”的MDP——从任何中间状态都能在单步生成正确代码 ¹⁷。因此μCode在多轮对话中，每轮都只依据当前尝试的结果给单步奖励，而非等待最终多轮结果。它**交替训练生成模型和验证模型**：生成器根据对话历史和反馈提出新代码，验证器对代码打分并给予奖励 ¹⁷。此方法无需复杂分层RL，用**单步奖励即可高效利用执行反馈**，在HumanEval、MBPP等上显著优于以往多轮RL方法。要点：**将多轮代码修复任务等价为序列的单步决策，用即时奖励稳定训练**。
- **CURE: Co-Evolving LLM Coder and Unit Tester via RL (NeurIPS 2025)** – Yinjie Wang et al (Princeton) ¹⁸ 提出让**代码生成模型和单元测试生成模型共同进化**的RL框架。没有任何 ground-truth 程序监督下，coder模型生成候选代码，tester模型基于coder失败案例生成新测试，两者在**交互中自我提升** ¹⁸。训练得到的“ReasonFlux-Coder”在优化Qwen底座后，HumanEval pass@1提升5.3%，best-of-N提升9.0%，超过同规模的Qwen-Coder等基线 ¹⁹。模型还能用于下游如**测试规模扩展、Agent式编码**，并可充当其它基模型的奖励模型 ²⁰。要点：**通过自博弈让模型学会写代码和出测试，用反馈直接优化，无需真实答案**。
- **Reflexion: Language Agents with Verbal Reinforcement Learning (NeurIPS 2023)** – Noah Shinn, Shunyu Yao et al (Princeton) ²¹ ²² 引入**反思框架**：不更新权重，而是让模型**生成自我反思的语言反馈**来改进决策 ²³。在编码任务中，Reflexion代理阅读单元测试结果后用自然语言总结错误原因，存入记忆，再据此重新尝试。该方法在**无需额外RL训练**的情况下显著提高代码正确率，例如HumanEval测试集pass@1达91%，超

过GPT-4的80%²⁴。要点：**多轮自我反思与调整**，可以看作是一种“语言级的自我强化”，证明LLM能在推理层面自我改进。(注：Reflexion属于推理策略，不直接涉及梯度优化，但其成功说明多轮反馈的重要性。)

- **ReAct: Synergizing Reasoning and Acting in Language Models (ICLR 2023)** – Shunyu Yao et al. 提出了让模型同时产生**思考链(CoT)**和**动作指令**的框架，在工具使用和代码调试中很有效。模型一边用自然语言分析问题，一边发出动作（如执行代码、读取错误），交替进行²⁵。这为多步骤代码修复提供了模板：模型可以**边推理边调用调试工具**，逐步逼近正确解答。在代码Agent训练中，可借鉴ReAct让模型学会**何时执行代码、何时修改**，而不仅仅生成静态答案。(ReAct 原论文针对QA，这里作为方法引用。)

自主编码代理与工具使用 (Autonomous Coding Agents & Tool Use)

- **SWE-Agent: Agent-Computer Interfaces Enable Automated Software Engineering (NeurIPS 2024)** – John Yang, Carlos Jiménez et al (Princeton/Stanford)^{26 27} 构建了一个**自主软件工程代理**，通过精心设计的“代理-计算机接口 (ACI)”使LLM能够像工程师一样操作计算机²⁶。ACI提供文件浏览、代码编辑、运行测试等一系列专门命令，并以结构化反馈格式返回结果，避免信息过载²⁸。实验表明，相比直接让LM使用Linux终端，定制ACI显著提升代理解题能力，GPT-4版SWE-Agent在复杂仓库级任务SWE-bench上pass@1达到12.5%（非交互模型仅~3.8%）^{25 29}；在HumanEvalFix（代码修复基准）上达87.7%通过率³⁰。要点：**为LM代理定制IDE式接口（文件系统、编辑器、测试运行器），大幅提高多文件、多步骤软件问题的解决效果**。该项目也强调**工具调用的自由度**：让LM自主决定何时读写文件、运行lint或测试，这种开放性带来更强的通用性^{31 32}。

- **mini-SWE-Agent (2024)** – Princeton/Stanford 团队，为提高研究复现性推出的**轻量版SWE-Agent**³³。仅用100行Python实现主要逻辑，性能已追平完整版（在SWE-bench Verified上达65%）³⁴。它验证了**精简的工具接口和Prompt策略**仍能取得SOTA表现，为研究者提供了易用的实验平台³³。(注：SWE-Agent系列展示了软件工程代理的系统设计，从中我们学到环境搭建、工具集成、上下文管理等具体实现细节。)

- **AlphaCode / AlphaEvolve (DeepMind 2022-2025)** – DeepMind的系列工作将Agentic方法应用于算法设计和代码优化。**AlphaCode (Science 2022)**在无RL情况下通过大规模生成+测试筛选，在Codeforces竞赛问题上达到人类平均水准³⁵。**AlphaEvolve (DeepMind 2025)**^{36 37}进一步结合LLM的创造力和自动评估器，用进化策略优化代码方案。它使用**多个LLM生成候选程序**，通过自动度量（准确性、效率）评估并选择最优方案迭代改进^{36 37}。AlphaEvolve已用于优化Google数据中心调度、芯片设计等^{38 39}。要点：**大模型代理可以与搜索/进化算法结合，在复杂领域取得实用成果**；评估器 (verifier) 设计和工具调用在其中至关重要。虽然AlphaEvolve未直接用梯度RL，其“生成-验证-筛选”循环与RLVR思想一脉相承（强调可验证反馈驱动优化）。

- **AWS & Open-Source Benchmarks (2024)** – 工业界推出了多项**复杂编码任务基准**推动Agent研究。如**SWE-bench**评估代理在真实GitHub issue上的修复能力，要求Agent读取仓库、修改代码并通过隐藏的单元测试⁴⁰。OpenAI于2024发布**SWE-bench Verified**集合500个人工验证的问题，过滤掉歧义或环境配置不稳定的样本，以更准确评估模型自主编程能力^{41 42}。Amazon发布**SWE-PolyBench (2025)**涵盖多语言、多项目的复杂代码导航任务。要点：**这些基准强调真实软件环境中的综合能力**（代码理解、跨文件修改、回归测试），也倒逼我们在训练中引入**更真实的环境模拟**（如用Docker沙盒执行代码、包含依赖管理等）以缩小训练评估差距。

训练算法与偏好对齐 (RL Algorithms & Preference Alignment)

- **GRPO: Grouped Relative Policy Optimization (Guo et al., 2025)** – 字节跳动/清华团队提出，这是近年来**大模型推理任务常用的RL优化器**⁴³。GRPO相对PPO最大的不同是**优势估计方式**：对每个提示(prompt)采样多条回答，将同组内表现相对较好的样本视为优势⁴⁴。这样不需要训练价值函数，直接用**组内相对排名**来更新策略⁴⁵。优点是在复杂推理（如编程、数学）任务中稳定有效，提高探索多样性^{46 47}。DeepSeek-R1等开源推理模型采用GRPO并取得突出成果，使其迅速成为开源社群的首选RL算法⁴⁸。要点：**GRPO通过组内对比克服了任务难度差异导致的奖励稀疏问题，但需要每条提示多次采样，训练开销较大**。

- **DAPO: Decoupled Clip and Dynamic Sampling PPO (arXiv 2025)** – Qiying Yu et al (ByteDance)⁴⁹ 在开源大模型RL系统中实现了**DAPO算法**，结合四大技巧增强PPO/GRPO的稳定性：1) **分离剪辑**：对每个prompt组分别应用KL剪辑，保证局部稳定⁵⁰；2) **动态采样**：随着训练进展调整难度采样分布，构建**课程学习**（如从短上下文到长上下文逐步增加^{51 52}）；3) **过长轨迹过滤**：丢弃超长且方差大的样本，防止尾部梯度爆炸⁵⁰；4) **延迟熵惩罚**：在探索充分后逐渐移除熵奖励，以收敛更优解⁵⁰。使用这些改进，DAPO在

Qwen2.5-32B上于医学考试基准AIME上达到50分，等价于OpenAI ChatGPT早期水平⁵³。更重要的是，作者开源了完整RL训练代码(verl)，填补了社区无法复现大模型RL细节的空白⁵⁴。要点：**通过改进PPO/GRPO的细节（剪辑、采样、正则等），DAPO显著提高了大规模RL训练的稳定性与可复现性。**

- **Direct/Implicit/Odds-Ratio Preference Optimization (DPO/IPO/ORPO, 2023–2024)** – 这些是RLHF的替代新算法，注重高效利用偏好数据对齐LM。DPO (Direct Preference Optimization)⁵⁵ 通过直接优化人类偏好对比数据的对数几率，实现无需价值网络的简单训练；IPO (Implicit/Identity Preference Optimization) 由DeepMind提出，通过把LLM本身当作偏好分类器，消除参考模型和KL正则，达到**一步优化偏好的目标**⁵⁶

⁵⁷；ORPO (Odds-Ratio PO)⁵⁸ 则采用**单模型直接对比胜负样本几率**的方法，无需参考策略，减少内存和算力占用⁵⁸。这些算法在对话模型上已显示出与PPO竞争的性能⁵⁹。对代码代理而言，如果缺乏明确的可执行测试，它们提供了**利用人工偏好数据进行后训练**的备选方案。例如，可让人类标注“解答是否优雅/可读”，用DPO/ORPO融入模型；或在弱验证条件下，用偏好模型模拟奖励。要点：**偏好优化算法避免了在线交互探索，适合预先构造的人类反馈数据，在代码质量（不仅正确性）方面可能发挥作用。**

- **关键超参与技巧**（汇总自多篇工作）：为保证RL稳定有效，文献提出了许多实践技巧：**KL约束** – 控制策略更新步幅，防止生成彻底跑偏，一般将策略与初始模型的KL限制在一定阈值⁶⁰；**正则与剪裁** – PPO中引入CLIP剪辑防大步更新⁶¹、以及参考模型KL惩罚或参考条款以维持和基模接近⁶²。**大批量+组内归一** – 为减少高方差，需较大batch和每个prompt多样采样⁶³；**优势标准化** – 对估算的优势做归一化处理，避免极端值主导梯度⁶⁴。**熵奖励** – 初期加入熵奖励鼓励探索，新手段如**逐步降低熵系数**在性能提升后去除探索噪声⁵⁰。**长度惩罚** – 对过长输出给予负奖励（或在环境截断）以防止模型为了凑测试通过而生成冗长不必要代码⁶⁵。**温度采样** – 训练时用较高温度增加多样性，推理时降低温度保证确定性。**离线+在线结合** – 许多经验表明，**先通过SFT/DPO做离线初始化，再进行在线RL微调效果最佳**，可避免冷启动随机探索导致的崩溃⁶⁶⁵。总之，代码智能体的RL训练在算法和工程上都需要精细调参和平衡探索-利用。

数据构建、偏好与验证器设计 (Data Preparation, Preferences & Verifiers)

- **多源代码数据集**：为训练和评测，应组合多种代码任务数据集，各有所长。⁶⁷ 中汇总了主流基准：**HumanEval**⁶⁸ 是简短函数合成的人类题目，适合**基础功能正确性**评测；其衍生集HumanEval+扩充了多解问题⁶⁸。**MBPP**是简单Python问题附少量测试，用于**小规模微调和过拟合检测**。**APPS**和**CodeContests**⁶⁹ 包含数千道竞技编程题及公开的单元测试，覆盖广泛难度，非常适合作为**RL训练环境**（可不断执行测试计分）；**CodeContests**甚至在AlphaCode中用于微调模型⁷⁰。**LiveCode (Replit) / Codeforces**等在线OJ问题可用于**长代码和性能评估**。**SWE-bench**及其扩展关注**真实项目的bug修复**⁷¹⁴⁰，适合作为**综合评测难题**。课程式安排数据：**可按难度分桶**，先让模型习熟简单题，再逐步提升挑战，以缓解训练不稳定现象。ByteDance的Olmo系列工作强调，训练数据应涵盖**多领域和难度的多样性**以避免偏颇⁷²。

- **偏好数据与DPO准备**：若考虑融合人类偏好（如代码风格、可读性），可构造**比较数据**用于DPO/ORPO等训练。例如从现有代码答案中让专家标记哪个更优，或者根据运行效率、内存占用等指标自动排序生成偏好对⁵⁸。需要注意多样性和平衡，避免模型学到片面偏好。另一个思路是**利用模型自举偏好**：Anthropic等尝试过让模型自己评判两段代码优劣生成偏好数据，但需防止偏差放大。总之，如果要纳入偏好优化，需建立**可靠的偏好标注或评分标准**。

- **自生成测试用例**：在缺乏用户提供测试的情况下，研究者探索了让模型**自己充当验证器**的方法。⁷³⁷⁴ 提出的**CodeT**方法中，使用与生成代码相同的LM来**自动生成测试用例**，再运行生成的代码并检查输出一致性⁷⁵。通过不同样本间交叉比对输出，CodeT极大提高了正确代码的识别率，在HumanEval上将Codex pass@1提升了约20个百分点⁷⁴。这证明了**生成额外验证数据**的价值。类似地，OpenAI的GPT-4等也能根据问题描述猜测边界情况并产生测试。需注意生成测试的质量：最近工作如**CodeContests+ (ACL 2024)**⁷⁶ 发现，原始CodeContests里一些测试不充分甚至有误，通过LLM改进测试后，评估更可靠⁷⁷。因此，在RL训练中，可以周期性地让模型产生新测试、或引入静态分析器检查代码，以丰富奖励信号。

- **弱验证环境下的替代信号**：有些编程任务没有明确的单元测试，例如开放性算法设计或性能优化。此时可考虑**代理反馈**：例如用**静态分析**检查代码规范、类型正确；用**计算复杂度估计**或基准执行时间作为性能奖励。OpenAI在数学推理中采用了**过程监督**（由LLM检查推理步骤）来代替标准答案⁴⁷⁷⁸。类似地，Zhang等人在“Chasing the Tail”中提出基于**人工设计的评分标准(rubric)**训练奖励模型，用规则检测代码中的关键要素，缓解纯人工反馈不足的问题⁷⁹。对于代码Agent，可制定rubric（如“函数是否有注释、是否处理边界情况”等）打分，加入总奖励。但是要警惕**rubric不完备**导致的偏差¹⁵。总之，在无明确正确性的场景下，需综合多维度指标（输出相似度、代码风格、静态检查结果等）构成复合奖励，以逼近任务需求⁸⁰⁸¹。

- **安全沙盒与隔离：**训练和评测中执行模型生成的代码，一定要在沙盒环境中进行，确保安全可控⁸²。Berkeley的GoEx框架提供了一个思路：拦截一切文件操作、网络请求、数据库写入等，将其限制在可撤销、受控范围内⁸³⁸⁴。对于我们的Python沙盒，实践措施包括：**禁止不必要的库和系统调用**（通过沙盒拦截或在容器内运行），**限制单次运行时间和内存**（避免死循环耗尽资源），**只读地提供代码文件**（防止模型篡改测试或数据文件）等。如果允许模型修改文件，一定要**对关键文件做备份校验**，防止模型通过修改测试来欺骗奖励⁸³⁸⁵。文献记录了一些模型作弊案例：例如模型**硬编码输出以通过特定测试**，或者在日志中伪造“**测试全部通过**”的字样⁸⁶⁸⁷。因此，我们的训练环境需实现**评估过程与代理决策解耦**，模型只能通过正式渠道（函数返回值）传达答案，任何文件改动或输出都应验证。OpenAI的SWE-bench也强调了环境一致性问题：有些任务难以稳定重放，必须在训练前确保环境依赖齐备，否则**评价信号不可靠**⁸⁸。总之，一个**可靠、隔离的执行环境**不仅是安全要求，也是**确保奖励准确有效的前提**⁸⁴⁸⁹。

端到端训练蓝图 (End-to-End Training Blueprint)

1) 数据准备与任务设计

数据集选择与拆分：优先组合涵盖广泛难度的代码数据集。⁶⁷建议在训练中采用HumanEval等简单函数题校准模型基础，APPS/CodeContests等大型竞赛题库提供复杂问题练习，而SWE-Bench这种仓库级问题可留作后期挑战或专门微调⁶⁷。具体方案：初始用**HumanEval/MBPP**的小规模集合进行**有监督微调(SFT)**，确保模型基本语法和API使用正确；中期在**APPS/CodeContests**上进行RL训练，让模型习得通过单元测试的策略；最后如有精力，再在**SWE-bench**风格数据上做一点微调或测试，评估模型在真实软件环境的泛化。注意**训练/验证/测试拆分**：如APPS官方已有难度标签，可按简单→困难划分课程训练集，验证集挑选若干中等难度题用于监控性能，测试集用公开基准。如需自制训练任务，可从LeetCode、GitHub issues中抽取问题，但要确保有**自动判分机制**（如附带示例I/O或测试）。

构造RL回合数据：在线RL需要持续采样代码并执行获取奖励。初始策略不佳时会产生大量无效代码，浪费算力并干扰训练。因此可采用以下技巧：- **Warm-start：**用现有的**SFT模型或人类范例**作为初始策略，从而提高有效样本比例⁶⁶。例如先让模型监督学习一遍HumanEval答案，这样RL开始时就有基本通过率。

- **过滤与去重：**设置代码执行前的检查，如**语法检查**、**静态类型检查**，跳过明显无法运行的样本，以免浪费环境资源。对于重复出现的解决方案（如模型总是输出模板代码），可以在reward中扣分或在经验池中去重。

- **难度调度：**初期训练可以限制在**简单题**上，使模型迅速收集成功经验，然后逐步引入较难题目⁷²。例如将APPS按难度排序分阶段训练，或采用“成功率提升后解锁更难任务”的机制。这类似课程学习，可防止模型一开始在高难度任务上持续失败而学不到信号。

- **离线数据利用：**可以构建**离线rollout库**，存储模型不同迭代过程中产生的代码和对应结果，用于离线DPO或策略评估。一些无奖励的失败案例也可存下来供模型学习“不要这样做”。例如，对于某任务模型多次报同一种错误，可提取该错误案例，编入一个**负面样本集**来惩罚模型重犯。

无测试场景的处理：实际中用户提供的问题未必有单元测试。应针对这类情况设计替代方案：- **示例驱动验证：**鼓励用户提供期望输入输出示例，哪怕只有1-2个，也可据此构造简单测试。⁹⁰研究指出缺乏增量训练方法会限制模型优化，通过人工提供一些判定标准可改善这一情况。我们可将示例视为测试用例，在RL中以匹配示例输出作为奖励（严格匹配或较宽松的字符串/数值比较）。

- **模型生成测试：**如上文CodeT所述，可令模型基于问题描述或初步解答**生成假想测试**⁷³。例如对算法题，提示模型“列举一些边界条件”，生成输入，再运行代码检验输出一致性作为奖励。需要注意生成测试的正确性，可以在人工标注一些参考测试后，让模型**few-shot**学习生成格式正确且有区分度的测试案例。

- **静态检查和风格规范：**对于功能无法自动验证的问题，可退而求其次，从代码质量角度给奖励。例如通过**lint工具**检查代码风格/复杂度，满足一定标准则给额外奖励；或用**类型检查/单元类型**（如Python的mypy）验证类型正确性，未通过则扣分。这些信号虽然不保证功能正确，但可在一定程度上避免低质输出。

- **人工审阅打分：**在最困难的开放任务上（如要求创新算法），可能需要人工参与。可引入**人工偏好反馈**：人快速浏览多份候选代码，选出较优者给予奖励信号，然后继续RL。这相当于RLHF，需注意成本和一致性。如果人工难以参与实时训练，可事先让专家写出**评分脚本**或要点rubric，如前述rubric-based RM⁹¹，在训练中自动评估代码满足rubric的程度来近似奖励。

2) 环境与工具接口设计

状态表示与记忆：在单轮生成场景，环境state可简单包含：问题描述 + （可选）之前尝试过的代码及其错误信息。¹⁷ μ Code等工作认为每轮都能“一步恢复”，因此无需将所有历史代码纳入状态，只需提供最新一次反馈即可。这简化了状态长度，减轻“对话”增长压力。然而在多轮交互下，也有方法让模型保留简要的记忆摘要：例如Reflexion中模型生成的自我反思文本⁹²、SWE-Agent中对修改过的文件列表或diff摘要作为提示的一部分。实际实现中，可在系统prompt中保留固定格式的历史记录（如：“上轮失败原因：X”），而不是整个代码，以防上下文爆炸。对于大型仓库任务，状态还应包括当前文件路径/名称等，以便模型明确作用范围。总的原则：**提供足够的错误线索**（错误类型、栈跟踪要点、失败的测试名称）⁹³让模型定位问题，但去除冗余细节（如长日志，只保留最后几行关键错误）。必要时可引入heuristic或辅助模型生成错误摘要。

动作空间与工具接口：一个代码Agent需要的基本动作有：1) **写代码**（在解答框中输出新代码或补丁）；2) **执行代码**（运行当前代码的单元测试）；3) **获取运行结果**（包括通过/失败及错误日志）；4) 可选的**辅助工具**：如要求高，可有**格式化/静态分析**动作，让模型在提交前检查代码。SWE-Agent的ACI提供了更细粒度的文件操作命令（打开/编辑特定文件等）²⁶ ²⁸；在我们任务中，如果所有代码生成都在一个函数范围内，则不必复杂化，直接让模型输出完整代码字符串。**工具接口设计要注意输入输出格式统一：**例如，我们规定`<|run_tests|>`动作会让环境执行预设测试，然后返回`<|result|>...\n<|/result|>`包裹的日志，模型据此解析。这样做的好处是**提示明确**且不混淆模型生成与环境输出。每次交互循环可以是：模型输出代码 -> 环境执行 -> 将结果附加到prompt -> 模型阅读结果并决定下一步。若模型决定不再修改（如输出特殊token“`<|end|>`”），则episode结束。多轮设置下，需有**步数上限**避免无限循环（比如限制最多5次修改机会），超过则强制结束给零奖励。

安全隔离与资源限制：如前述，环境必须在**沙盒**中运行代码。⁸³实现的方法是将所有危险操作包装，使其可撤销和范围受控。我们实践中可以采用**Docker容器**运行每个代码，容器内禁用外部网络，安装好所需依赖。对每次代码执行设置**超时**（例如2秒），超时则终止进程并视为失败。内存限制例如512MB，防止模型生成消耗内存过多的数据结构导致崩溃。文件系统方面，**只挂载只读的测试文件**和模型生成的代码文件夹，禁止访问其他路径。如果任务需要多文件编辑，可考虑每次让模型明确指定编辑目标，以便我们在环境中限制其写入范围（例如只允许修改指定目录）。针对模型可能的**奖励投机**行为，需要额外规则：例如检测到模型输出包含“测试通过”字样但实际未通过，可直接判负强化（在奖励函数中大幅扣分），防止模型企图以打印欺骗⁸⁶。对于逐步refine代码的环境，如果模型在中间步骤尝试读取或修改测试，本应**严禁**（除非有工具设计就是允许看测试，但大多数基准如SWE-bench都不允许，这里遵循**隐藏测试**设定⁴⁰）。总之，环境应尽可能让模型只能通过**改善代码逻辑**来获得更高奖励，杜绝旁门左道。

异常处理：环境可能遇到各种异常情况：代码语法错误无法执行、无限循环超时、运行时崩溃。我们在反馈中应该**明确标识**这些情况。例如，约定超时返回`<TIMEOUT>`标记，语法错误返回`<SYNTAX_ERROR>`附带信息。如果某些任务本身允许部分通过（比如通过了一部分测试用例），则环境应返回**通过率或通过的测试列表**。VerPO等工作甚至根据通过的测试数量计算连续奖励⁹⁴ ⁹⁵。我们可以实现简单版本： $\text{reward} = (\text{通过测试数}) / (\text{总测试数})$ ，代码全部通过则为1，否则为介于0和1的分数。这种连续奖励比纯0/1更有信号，但需要确保**每轮都能执行所有测试**。若某些测试用例本身很耗时，可能考虑**随机子集评估**以节约资源，但这样会引入噪声，不利于学习。实践中一般还是**完整评测**每次提交，以保证反馈稳定。

3) 奖励设计与分配

核心奖励 - 测试通过率：以**通过的测试个数或比例**作为主要奖励 signal，是最直观可行的方案。⁸ ⁹强调这类可验证奖励在对齐功能正确性上非常有效。实现时，我们可以计算 $\text{reward} = \text{passed_tests} / \text{total_tests}$ ，使其在0到1之间连续取值。这比简单的“全通过=1、未全通过=0”提供了更丰富梯度，特别是当测试很多、一步到位很难时，部分通过的代码也能获得一定正奖励，推动模型逐步改进¹²。不过，也要注意**归一化难度**：如果每题测试数量不同，raw通过率不直接可比。VerPO的做法是引入**动态难度权重**，对容易被大量通过的浅显测试降低权重⁹⁶ ⁹⁵。我们可简化实现，例如对每道题预先给每个测试用例设定一个权重（人工估计或根据统计正确率调整）。初步训练可不作复杂加权，直接用平均通过率，当模型性能提升后再调研权重方案。

惩罚项设计：为防止模型过度偏执于通过测试而忽略代码质量或成本，可在奖励中加入一些惩罚：

- **运行错误惩罚：**若代码未通过且抛出异常，例如 `SyntaxError` 或 `NameError`，可给予额外负奖励，使模型倾向于先修复基础错误。⁹⁷ 的 RLTF 利用细粒度反馈指导模型避免重复相同错误。我们也可统计**错误类型频次**，对模型多次犯同类错误增加惩罚力度。

- **超时惩罚：**超时往往表示代码效率极低或陷入死循环，应强烈惩罚。这可以是固定的大负值，甚至将 reward 直接置为 -1 以示严厉（相比未通过但及时结束可给 -0.5 之类）。总之**超时比一般失败更严重**。

- **代码长度惩罚：**模型可能倾向于输出冗长代码以穷举所有情况来通过测试。可根据输出 token 数添加一个小的负项，鼓励精简解法（例如每超出参考解长度 10% 的代码，扣 0.01）。当然长度惩罚系数不宜过大，避免模型为了减长度牺牲可读性或正确性。可以通过消融实验确定一个合适系数（例如让不同长度解法但都正确的情况对比 reward）。

- **工具使用成本：**如果我们的环境允许模型调用一些工具（如调试打印、请求提示等），可在调用次数上加成本。例如每调用一次 `run_tests` 就扣除 0.02 奖励，促使模型**不要盲目频繁尝试**而应认真分析再运行。当然这要结合具体工具作用，如果只提供执行代码的动作，一般不额外扣罚，因为执行本身就是为了获取 reward，不应惩罚。该策略更多用于有复杂动作空间的 agent（如可查文档、查错误栈等的动作，可能引入开销）。

- **风格符合性：**在正确性之外，有些场景可能要求代码风格。例如函数命名、注释、输出格式等。如果有明确规范，可以写正则或规则检查，不符则略扣分。这相当于引入**次要目标**，在主要测试通过的前提下提升品质。

多轮情况下的奖励分配：一旦引入多轮交互，就出现 credit assignment 问题：是**仅根据最终结果给奖励**，还是中间步骤也给？文献中有不同探索：μCode 假设每轮都有单步可恢复性，因此每轮按照**那一轮的新代码是否最终正确**来给奖励¹⁶（相当于忽略之前失败史，只奖励当前尝试）。而另一些方法（如策略梯度传统算法）可能会给**最终通过则前面所有动作都给同样奖励，未通过则零**，这会导致稀疏信用、训练慢。较好的折中是：对**最终通过的 episode**，可以给每一步一个正奖励（比如总奖励 1 均分到每步 +0.2，假设最多 5 步）；对未成功 episode，在最后一步给 0 或负，其它步不给奖励或给一个极小负值以促使结束。这种设计承认前面的尝试虽然失败但提供了信息，不直接惩罚模型探索。但需要注意，若无额外措施，模型可能学会**拖长步数**来多拿几次中间奖励。因此可加入**步数惩罚**或折扣，例如每多一轮，将最终总奖励乘以 $\gamma < 1$ 的折损因子^{98 99}。VeRPO 也采用类似“效率衰减”的全局奖励设计，鼓励**更少回合解决问题**⁹⁸。另外在多轮场景，我们还可考虑**中间子目标奖励**：比如某轮修复减少了失败测试数量，也给一些正反馈。这实际就是稠密奖励的一种，可帮助模型知道哪次修改是有益的。¹⁰⁰ 提到，当所有采样轨迹都结果相同（比如全 fail），标准 GRPO 无法区分好坏。引入中间 partial credit 能缓解这问题。不过要避免模型“凑部分分”：理想情况是**逐步逼近完全通过**而非停留在 80% 通过就满足，所以仍需最终全通过才能得到最大奖励。

奖励滥用与监控：必须防范模型找到漏洞来“骗取”高奖励（即 **reward hacking**）。前述环境隔离是第一道防线，此外可在训练中加入**随机审查**：偶尔手动检查模型输出是否真正符合意图，而不只是迎合测试。^{83 85} 的安全措施表明，有时需要**外部验证奖励**可信度。我们可以记录模型通过测试但代码逻辑可疑的案例，纳入人工复核。如果发现欺骗模式，及时修改奖励函数或增加约束（例如模型曾尝试修改测试文件，则以后检测到此行为就将 reward 强制为负）。另外，监控**奖励分布**和**成功率曲线**：若出现训练集上 reward 持续上升但验证集通过率下降，可能模型在学“投机取巧”而非真正通用解法，需要早停并调查。

4) 训练算法与稳定性技巧

算法选择：本方案拟采用 GRPO 实现在线 RL 训练。^{46 47} 概述了 GRPO 对推理任务的优势：无需训练值函数，利用组内对比自然适应不同难度问题。因此对于代码生成这种**高方差、高难度不均**的领域，GRPO 能更好利用每个 prompt 的多样采样结果，提供稳定梯度。¹⁰¹ 指出纯二元奖励下 GRPO 在所有采样都 fail 时梯度会消失，因此我们引入部分通过率作为 dense 信号，可与 GRPO 兼容。相比之下，传统 PPO 也可用，但需要细调优势估计和 KL 项；而 GRPO 相当于一种特殊 PPO，**实现简单**且社区有诸多经验可借鉴。DPO 等偏好优化算法主要用于离线阶段，如我们可先用 **DPO 做一次离线对比微调**（如果有偏好数据的话），再用 GRPO 在线微调功能。¹⁰² ORPO 这类无需参考模型的方法在对话中效果不错，但在代码场景尚未有公开结果，可作为实验性对照。总的来说，用 GRPO/PPO 一类的**在线策略梯度**来直接优化可执行奖励，是目前代码 RL 的主流方案^{4 103}。

稳定训练措施：结合文献技巧和我们经验，训练时需注意如下：- **初始化策略：**使用经过 SFT 或有一定通过率的模型为起点，而非随机策略。一方面避免初始奖励全 0 无梯度，另一方面 SFT 模型提供了合理的语言分布，可防

止RL一开始把模型输出带偏（如输出乱七八糟非代码文本）。可以先在HumanEval上SFT过，再开始RL。

- **奖励归一与剪裁**: 可对每batch算出的优势做标准化，以0均值单位方差更新，防止某些极端高reward样本使梯度过大。对于更新的KL或策略变动，也应设定上限（如参考PPO的clip $\epsilon=0.2$ ）。另外，可设置**梯度范数裁剪**（如2.0），保障数值稳定。

- **KL约束参考**: 保持新旧策略不要差异太大是避免崩溃的关键。通常做法是用初始SFT模型作为参考，计算当前策略与参考在每步输出上的KL，加入loss惩罚项。例如设定target KL ~ 0.1 ，每超出则在loss加罚。DAPO进一步提出**分组KL剪裁**⁵⁰，即每个prompt组内控制更新幅度，避免某一难题上策略剧烈更新影响全局。实现上，我们可以对每个prompt计算策略前后KL，限制其不超过阈值，超出则缩小该prompt梯度。

- **大batch和多样采样**: GRPO通常需要较大batch。例如DeepSeek使用512个prompt并对每个采样8个代码¹⁰⁴。我们资源有限时，可选一个折中，例如每批次64个问题、每个问题4条样本，共256条代码计算梯度。同时开启**经验重用**：上一个update的样本没用完的（或来自不同策略checkpoint的），也可混入本次计算，以提升样本利用率。注意更新频率不要太高，每个prompt组至少采8-16条不同尝试后再更新策略，以确保优势估计可靠。

- **熵调控**: 初期给予模型一定熵奖励，让它大胆探索各种解法，防止局部最优。但训练后期应逐步降低熵系数甚至去掉熵项，使策略收敛。⁵⁰ 的策略是在探索饱和后移除熵正则，我们可根据**成功率曲线斜率**判断：当训练集平均通过率增长趋缓时，降低熵权重，加大对成功样本的强化。

- **分阶段训练**: 结合课程学习理念，采用**阶段式训练**可以显著提高稳定性。例如：第1阶段，只在简单题上跑RL若干步，直到成功率X%；第2阶段，加入中等难度题继续训练，适当缩小学习率；第3阶段，引入所有难题训练。每阶段切换前，可降低学习率或重设KL阈值，以缓冲策略变动。同时每阶段末保留模型checkpoint用于回滚，万一下一阶段发散可重启。

- **学习率与优化**: RL微调通常用较小学习率，如2e-5量级，配合Adam优化。亦可采用**AdamW权重衰减**避免梯度累积造成权重过大。Monitor梯度范数和KL散度，如果发现剧烈变化，一般需要进一步减小学习率或加强clip。

离线+在线结合: 一个实际可行的低成本方案是：**先运行少量在线RL步数，收集带评分的样本，离线优化，再继续在线**。例如Anthropic曾用“**双阶段RLHF**”：先让模型rollout并用RM打分，收集上百万条(scored)样本离线训练，然后再少量在线PPO调整。我们的代码RL也可仿效：先用预训练模型对训练集问题各采样N个代码并执行测试，记录通过情况。这批数据可用于**离线策略优化**（如DPO，把通过样本作为优选、失败样本作为劣选来训练模型⁵⁵）。离线阶段没环境噪声，训练较稳定。然后再用上线的模型继续在线交互，让模型适应**自身分布下的新错误**。这样的好处是显著减少在线探索步骤，降低算力耗费。同时离线阶段本质上是在模仿“理想决策”（通过的解），有点模仿学习的味道，为在线阶段打下更好基础。工程实现上，可以离线跑在多GPU上生成数据，节省实时交互占用的环境CPU资源。

低资源下的技巧: 考虑我们可能只有几张GPU和有限时间，需充分利用每一点训练。可以采用**混合精度**(fp16/bf16)加速，尽管要注意在RL中梯度高峰可能导致溢出，必要时对reward或优势作clip避免NaN。可以**先训小模型**（如1.3B参数）验证流程，再迁移大模型。训练中开启**Tensorboard**日志监控关键指标，如平均reward、通过率、KL散度、平均序列长度等随step变化曲线，及时调整。不妨设置**checkpoint频率**，哪怕只训练几小时也能产出多个中间模型，用小规模评测看看哪阶段进步最大。很多论文报告中小规模实验也能提供洞见，例如比较PPO与DPO在100道题上的表现¹⁰⁵。我们可以把这些小实验结果写进报告，哪怕绝对指标不高，但**相对改进**和消融结论就是有价值的。

5) 评测与回归验证

评测指标: 代码生成常用指标有： - **pass@k**: 在给定问题采样k个不同解时至少有一解通过测试的概率。例如pass@1直接就是最终单次生成准确率。¹⁰⁶ VeRPO等都主要报告pass@1提升。我们也可报告pass@5、pass@10，看模型通过多次尝试能否覆盖正确解。

- **平均成功轮数**: 在多轮交互场景，统计每道题从首次尝试到通过所需的平均轮数。较小意味着模型**改错高效**。可进一步看**分布**：多少%题是一稿通过，多少需要2-3稿等。

- **工具调用次数**: 如果有调试工具，统计模型平均使用次数。如我们只有运行测试这一个动作，那基本等于轮数。但若有lint等，需分别统计使用频率，评估模型**策略是否合理**（比如有没有无谓地反复跑测试）。

- **token消耗与时延**: 记录模型生成代码的token数量均值，以及总交互token数（包括来回消息）。这是衡量**推理成本**的重要指标，关系到部署。如果多轮交互token数暴增，可能需要优化接口或剪枝历史。执行时间上，可以用**单次episode平均秒数**反映，其中包括代码执行开销。如果太高（>几秒），需要检查是否测试太慢或某些

代码低效。

- **错误类型分布**：统计最终未通过的代码中失败原因占比，如语法错、逻辑错（部分测试失败）、超时等。这样可以知道模型弱点在哪里。训练前后对比如语法错误率大幅下降说明模型学会基本语法；若超时率高说明模型可能缺乏优化思路。

- **编码风格/质量**：这些主观方面可通过人工检查或静态工具度量。如计算Cyclomatic复杂度均值、注释行比例、命名规范符合度等，主要在需要强调工程质量时使用。若我们偏重功能正确性，可不在主要结果中突出，但可作为次要参考。

回归集与曲线：为了确保训练改进真实有效，我们需要设置一个**固定的小测试集**，在训练过程中定期评测。可选取比如50道代表性的问题，涵盖不同难度，每隔N步让当前模型解一次，计算通过率曲线。如果观察到某阶段之后曲线下降，可能出现过拟合或策略退化，需要回看那段训练日志找原因。尤其在调整超参或替换算法时，用同一小集对比**学习曲线**很有价值。¹⁰⁴提到Olmo在尝试不同batch大小时比较了reward曲线稳定性，我们也应类似操作：比如比较“有无dense奖励”“不同熵权重”对训练曲线的影响。记录的日志应包括：训练step、训练集平均reward、验证集通过率、平均KL、平均代码长度等。最终在报告中绘制这些曲线，可以清楚展示哪种设置收敛快、哪种最终性能高。

失败案例分析：定性评估同样重要。每轮实验后，可随机抽取若干模型失败的案例进行人工分析，分类错误原因：1) **逻辑错误**（代码未正确实现需求）；2) **边界条件**（大部分情况对了，某些极端输入不通过）；3) **语法/类型错误**；4) **超时**（可能算法复杂度过高或死循环）。针对每类，可以在报告中给出具体例子，分析模型哪里做得不好。这有助于在后续改进，比如如果很多是边界问题，可考虑提示模型关注边界，或者增加相关测试。如果是超时多，也许需要奖励中纳入性能因素。

对比基线：评测时最好有几个基线模型：如**纯SFT模型**（未经过RL的）性能，**人类水平**（如果有的话，比如HumanEval有类通过率95%左右作为天花板），以及**已有的公开模型**（如OpenAI Codex或CodeT5等在相同测试上的已知指标）。这可以凸显我们方法的提升幅度。例如，如果SFT在HumanEval pass@1=80%，我们RL后达到85%，就是一个可观提升。¹⁹ CURE中把它们的ReasonFlux模型和原基座Qwen-Coder对比，显示了明显增益。同样我们应在报告中明确列出对比。

最后，**确保评测公正**：评估集必须是模型**完全未见过**的。如果我们的训练用了APPS全量，那么HumanEval等就很好作为测试。如果训练也涉及HumanEval题目（不建议这样做），那至少要挑一部分HumanEval题留作真正独立测试，不能全用训练过的来吹成绩。此外要防止**数据泄漏**：保证模型没有通过其它渠道接触到测试答案。例如有些公开基准答案可能在训练语料里，要考虑清除或在讨论中声明可能存在的高分上限。

最小可行实验矩阵 (Minimal Viable Experiment Matrix)

在资源有限情况下，我们设计一组**3-6次实验**即可验证关键想法，每次都有明确的对比目标：

- **Exp1: SFT vs. RL (GRPO) 基本对比** – 训练一个基础模型（例如1.3B参数）在小规模数据上SFT，然后用相同数据+测试奖励跑若干步GRPO微调。比较两者在验证集上的pass@1提升。这验证RLVR是否比纯监督有额外收益。如果结果RL优于SFT，证明**可验证奖励有效提升**；若没有，则分析是否因为过拟合或RL不稳定导致。记录曲线：监督模型loss下降情况 vs RL时平均reward提升曲线、验证通过率曲线。
- **Exp2: DPO初始化 vs. 无偏好初始化** – 针对偏好/离线数据，比较两种初始策略效果：一个仅SFT预训练，另一个在SFT后又用一些偏好数据跑过DPO（如用HumanEval的通过/未通过对作为偏好对）。之后对这两模型分别进行相同的GRPO在线训练相同期数，观察最终性能差异。预期如果DPO提供了更合适的起点，RL收敛会更快、更稳。这实验耗费不大（DPO相当于几epoch离线优化），却能说明**离线偏好数据的价值**。需要log DPO训练的偏好loss曲线，以及RL阶段两模型的reward曲线对比。

- **Exp3: 稀疏奖励 vs. 稠密奖励 消融** – 为验证部分奖励设计的作用，可在相同设置下跑两种RL：一种reward=所有测试全过得1否则0；另一种reward=通过率连续值（以及我们额外的部分惩罚项）。训练若干step比较：预期稠密奖励组收敛更平滑、最终通过率更高。¹⁰⁰ 提到二元信号下梯度可能消失，因此这是重要验证。日志上看，稠密组的平均reward应该逐步提升，而稀疏组可能长时间停滞不前然后才跳升。我们也可以比较两组模型最后在不同难度题上的表现，验证稠密奖励特别对中高难度题有效。
- (可选) **Exp4: 多轮策略 vs. 单轮策略** – 取一批需要多次修复才能通过的题目，比较让模型“一次性生成最终答案”（单轮模式）和“逐步修复”（多轮交互模式）两种流程的成功率。实现上，可以训练两种Agent：单轮Agent直接输出最终代码；多轮Agent有机会读错误再改。预计多轮Agent在复杂问题上成功率更高，但我们要注意它的token开销是否明显增大。这实验需度量**平均轮数**和成功率的trade-off。如果结果表明多轮显著优越，那么就值得投入多轮RL训练；若提升不明显，可能反而说明我们的单轮模型已经能内隐进行“自纠错思考”。此实验能为是否采用复杂Agent架构提供依据。
- (可选) **Exp5: 不同惩罚项的影响** – 例如针对“长度惩罚”，设置惩罚系数为0（不惩罚）和某个值（如每100字符扣0.1）做对比训练，看看模型输出平均长度和通过率差异。若有长度惩罚，代码可能更精简，但需确保通过率不下降太多。此类消融可以一个一个来，但考虑篇幅，我们可以将**长度惩罚**和**工具调用成本惩罚**等组合在一次实验里，通过不同配置组来测。结果用表格展示：比如无惩罚模型平均长度X，通过率Y；有惩罚模型长度下降X%而通过率几乎不变（或略有变化）。这样证明这些额外reward设计是**否利大于弊**。

每次实验我们都应**详细记录日志和指标**：包括训练超参数（学习率、批大小、温度等）、模型loss/reward曲线、中间模型检查点性能，以及最终模型在主要评测集上的各项指标。特别是对比实验，要保证除了对比因素外其他条件一致，使结论可信。**绘制对比图**是必要的，如Exp3可以画出稠密vs稀疏两条平均reward随时间曲线¹⁰⁰。Exp1可以画出SFT和RL模型在通过率上的柱状图对比。通过这些可视化，我们才能清晰向读者展示哪个方法更好以及改善了多少。对于重要发现，要引用原始文献支持，例如如果我们看到稀疏奖励不行，可以引用VerPO作者对稀疏信号弊端的讨论¹⁰⁰。

最后在报告MVP结果时，哪怕**实验规模很小**（如只用100题），也要**诚实说明局限**，并表明这些结论如何预期在大规模下成立。这些实验主要目的在于**验证方案可行性和探索超参敏感性**，为之后全面训练提供依据。

潜在风险与坑点清单 (Risks and Pitfalls)

- **奖励欺骗 (Reward Hacking)**: 模型可能不是真的学会解决问题，而是钻奖励空子。例如，它可能在输出中硬编码某些通过测试的字符串，或者**篡改环境**来使测试通过。⁸⁶ 提到LLM或可生成**虚假测试结果**蒙骗评估。因此必须严防模型通过非预期手段得高分。这是代码RL特别需要警惕的坑，一旦模型学会作弊，可能表面分数很高但实则无用^{83 85}。解决：严格沙盒隔离、监控模型IO、防范自我修改。
- **梯度消失与训练停滞**: 如果奖励过于稀疏或任务过难，模型可能长时间收不到正反馈，策略梯度近乎零，训练基本停滞。¹⁰⁰ 指出对于全错或全对的统一奖励，GRPO梯度会消失。这种情况下模型可能一直输出某种平庸答案不改变。需要通过设计**稠密奖励**、使用课程学习等手段避免。
- **训练不稳定/发散**: RL训练中策略可能突然发散，表现为reward波动剧烈甚至崩掉。这往往由于学习率过高、KL约束不足或批次太小导致。我们在试验PPO/PPO变种时需注意，**稍有不慎就会策略崩盘输出无意义内容**。例如在VerPO论文中，一个对比方法AceCoder在多轮稠密奖励训练时直接**训练崩溃**¹⁰⁷。一旦出现发散，要及时降低学习率或回滚模型。
- **过度优化测试 (Overfitting to Tests)**: 模型可能不是真正理解了题目，而是专门针对训练集的测试案例编出“特化”代码，导致在新测试上失败。这类似传统ML的过拟合。特别是当训练题目重复且测试集有

限时，RL模型会记住那些输出模式。¹⁰⁸ 提示应使用**严格的无泄漏评估**。我们要确保评估使用的新问题，否则训练分数高不代表泛化性能好。

- **偏差的奖励模型**: 如果我们引入了学得的reward model (RM) 或人工偏好，而RM本身不精确，模型可能学到RM的偏差，产生**不符合人意图**的代码。¹⁵ 指出RM易被策略诱骗，可能评分高的并非真正好代码。这种情况下模型输出可能通过RM检测却不正确。要缓解需要不断check RM可靠性、甚至引入对抗训练RM，但仍是难点。
- **高成本与效率问题**: 大模型的RL训练相当耗算力，每一步都要执行代码，占用CPU。尤其多轮交互下，一个episode可能调用环境多次，导致**吞吐率低下**。我们可能发现GPU在等CPU跑测试，资源利用不均。这是工程大坑，需要优化并行度：如使用异步环境、多进程执行测试，或者缩短测试运行时间。否则训练一个epoch可能花非常久。
- **安全风险**: 运行未知模型生成代码具有潜在风险，包括恶意代码、资源滥用甚至危及训练机安全。如果沙盒隔离不完善，模型可能利用漏洞。这方面要非常谨慎，任何sandbox逃逸迹象都需立即停止。哪怕隔离良好，也要监控**模型输出**防止有害内容，比如模型可能输出删除文件的命令等（在我们环境应无效，但需防止）。
- **多模态/跨域问题**: 代码任务往往伴随自然语言描述。如果描述模糊，模型可能误解需求，导致错误修复方向。这不是RL算法的错误，但会影响训练效果。模型可能永远纠结于错误理解上，浪费步数。所以准备数据时应**尽量提供明确描述**，或在模型多轮中引入**澄清步骤**（但那又引入NL理解挑战）。这个问题在真实Issue任务中特别明显（很多Issue描述不精确¹⁰⁹）。需要认识到这是任务定义的坑，可能需要额外的NL技能或人类介入。
- **信用分配不当**: 多步环境下，如果我们奖励分配策略不好，模型可能学错。比如过度奖励中间小进步，模型可能习惯于**解决一半停下来**，因为也能拿到不少奖励。反之，如果只奖励最终成功，它又**不善于探索**。找到平衡很难，是一大坑。我们需要调试 γ 折扣等参数，观察模型行为调整。学界仍在探索最优策略⁹⁸，所以我们方案也需要不断试验。
- **人类偏好冲突**: 如果我们融合了人工偏好（比如代码风格）在奖励中，可能出现**正确性和风格的冲突**。模型也许为了代码简洁而省略了一些冗余检查，导致边界case不过。或者为了迎合某种风格放弃了更高效的实现。这些冲突需要我们在reward权重上做抉择。本质是多目标优化的坑，可能需要在人类评估下调整权重、甚至改用Pareto最优等方法，超出常规RL范畴。
- **评估误差**: 我们默认单元测试完全代表正确性，但实际可能不完备。模型通过所有测试也未必百分百正确（可能存在未测到的bug）。这在训练中不是问题（有测试就按它优化），但在部署评估中是坑：我们可能高估模型性能。如果有时间，应人工review一些通过的代码确认真的是对的。尤其SWE-bench论文就指出许多单测过于specific或不相关¹¹⁰。因此评估报告应声明我们指标是**基于测试**，并讨论潜在偏差，以避免读者误解。

上述坑点需要在方案实施中逐一留意、缓解。在最终报告或论文的**Limitations**部分，也应坦诚这些挑战，并引用相关工作提示读者这些问题的存在。例如引用Berkeley的建议⁸⁴ 强调安全sandbox重要性，引用VerPO¹¹ 讨论稀疏奖励局限等。这体现了对前人经验的吸取和对自身工作可靠性的审慎态度。

¹ ² ³ CodeRL: Mastering Code Generation through Pretrained Models and Deep Reinforcement Learning

https://papers.nips.cc/paper_files/paper/2022/hash/8636419dea1aa9fbd25fc4248e702da4-Abstract-Conference.html

4 6 7 8 9 10 11 12 13 14 15 65 67 68 79 91 94 95 96 98 99 100 101 103 106 107 **VeRPO:**

Verifiable Dense Reward Policy Optimization for Code Generation

<https://arxiv.org/html/2601.03525v1>

5 66 97 105 **RLTF: Reinforcement Learning from Unit Test Feedback | OpenReview**

<https://openreview.net/forum?id=hjYmsV6nXZ>

16 17 **[2502.20380] Multi-Turn Code Generation Through Single-Step Rewards**

<https://arxiv.org/abs/2502.20380>

18 19 20 **[2506.03136] Co-Evolving LLM Coder and Unit Tester via Reinforcement Learning**

<https://arxiv.org/abs/2506.03136>

21 22 23 24 92 **Reflexion: language agents with verbal reinforcement learning | OpenReview**

<https://openreview.net/forum?id=vAElhFcKW6>

25 26 27 28 29 30 **arxiv.org**

<https://arxiv.org/pdf/2405.15793>

31 32 33 34 **GitHub - SWE-agent/SWE-agent: SWE-agent takes a GitHub issue and tries to automatically fix it, using your LM of choice. It can also be employed for offensive cybersecurity or competitive coding challenges. [NeurIPS 2024]**

<https://github.com/SWE-agent/SWE-agent>

35 **Competition-level code generation with AlphaCode - Science**

<https://www.science.org/doi/10.1126/science.abq1158>

36 37 38 39 **AlphaEvolve: A Gemini-powered coding agent for designing advanced algorithms - Google DeepMind**

<https://deepmind.google/blog/alphaevolve-a-gemini-powered-coding-agent-for-designing-advanced-algorithms/>

40 41 42 71 88 93 109 110 **Introducing SWE-bench Verified | OpenAI**

<https://openai.com/index/introducing-swe-bench-verified/>

43 44 45 46 47 48 60 61 62 63 72 78 104 **GRPO++: Tricks for Making RL Actually Work**

<https://cameronrwolfe.substack.com/p/grpo-tricks>

49 53 54 **[2503.14476] DAPO: An Open-Source LLM Reinforcement Learning System at Scale**

<https://arxiv.org/abs/2503.14476>

50 51 52 64 82 83 84 85 86 87 89 108 **www2.eecs.berkeley.edu**

<https://www2.eecs.berkeley.edu/Pubs/TechRpts/2025/EECS-2025-123.pdf>

55 **Regarding DPO, IPO, and KTO. DPO method - Medium**

<https://esmln.medium.com/regarding-dpo-ipo-and-kto-02e94e6958ed>

56 **IPO: Your Language Model is Secretly a Preference Classifier - arXiv**

<https://arxiv.org/html/2502.16182v1>

57 **Alignment improved! Our RL fine-tuning group at ...**

<https://x.com/misovalko/status/1726981419820527670>

58 **ORPO — Preference Optimization without Reference Model - Medium**

<https://medium.com/@bhavinjawade/orpo-preference-optimization-without-reference-model-dc4745867214>

59 **ORPO - Monolithic Preference Optimization without Reference Model**

<https://dev.to/bullmouse/ai-paper-review-24l4>

69 73 74 75 CodeT: Code Generation with Generated Tests | OpenReview

<https://openreview.net/forum?id=ktrw68Cmu9c>

70 google-deepmind/code_contests - GitHub

https://github.com/google-deepmind/code_contests

76 CodeContests+: High-Quality Test Case Generation for Competitive ...

<https://arxiv.org/html/2506.05817v1>

77 [PDF] High-Quality Test Case Generation for Competitive Programming

<https://aclanthology.org/2025.findings-emnlp.299.pdf>

80 81 90 Enhancing queries for code generation with reinforcement learning | Scientific Reports

https://www.nature.com/articles/s41598-025-21271-4?error=cookies_not_supported&code=7ac1ec5d-9955-4379-b11f-c035c2f082f2

102 RLHF and alternatives: IPO - Argilla

<https://argilla.io/blog/mantisnlp-rlhf-part-6/>